

从零搭建实施手册

访客登记 · NFC 通行卡 · 链上审计证明

Spring Boot + MySQL

React + shadcn/ui

Windows NFC Reader Agent

Sui Move + Testnet Worker

部署、测试、备份与恢复

你自己构建，不是复制一套完整源码

每一步说明：做什么、放在哪里、为什么、如何验收。

代码为教学片段；需要你补全、运行并记录验证结果。

目录 / 搭建基础与 UI

按章节逐步构建；点击标题可跳转。页码是 PDF 实际页码。

00 如何使用这本搭建手册	4
01 全局路线与里程碑	5
02 架构、边界与必须记录的决策	6
03 Windows 开发环境安装	7
04 从 project structure 开始	8
05 初始化 Spring Boot 项目	9
06 本地 MySQL 与持久化	10
07 Backend 配置与第一个 endpoint	11
08 数据模型与 migration 顺序	12
09 约束与项目分层	13
10 API 契约先于页面	14
11 登录与角色：先选一种方式	15
12 Session、CSRF 与前端调用	16
13 四类访客登记：后端先完成	17
14 Hospital API adapter 与 MRN 校验	18
15 队列、审核与拒绝	19
16 卡片入库与发卡事务	20
17 幂等性、归还与逾期	21
18 审计、报表与管理员功能	22
19 初始化 React、Vite 与 shadcn/ui	23
20 前端目录、路由与请求层	24
21 红白直角设计系统	25
22 访客手机 UI：从二维码到 reference	26

目录 / 硬件、链上与上线

按章节逐步构建；点击标题可跳转。页码是 PDF 实际页码。

23 Staff / Admin UI 与扫描状态机	27
24 硬件准备：先确认卡，再选 reader	28
25 Windows 驱动与独立试读	29
26 卡片 provisioning 与类别解析	30
27 Reader Agent 与云端 Scan Job	31
28 Agent 打包与现场安装	32
29 Sui 的职责与证明数据	33
30 安装 Sui CLI 与 Testnet 钱包	34
31 Move package：先设计再写合约	35
32 Move 测试与发布	36
33 TypeScript Sui worker 与 outbox	37
34 上链确认、重试与篡改验证	38
35 部署拓扑与环境变量	39
36 Railway：Backend、MySQL、Worker	40
37 Vercel、同源 API 与 cookie	41
38 CI、版本发布与回滚	42
39 备份恢复、日志与故障处理	43
40 测试矩阵：按需求验收	44
41 常见问题排查顺序	45
42 最终演示、交付与自主学习顺序	46
A 代码片段索引与补全任务	47
B 官方参考与版本核对 (1)	48
C 官方参考与版本核对 (2)	49

00 如何使用这本搭建手册

这是一条面向单人 FYP 的完整实施路线：你负责建立文件、补全业务逻辑、运行测试和部署；本书提供顺序、接口约定、关键片段和验收标准。它不是可直接运行的整套源码，也不声称片段已经在你的硬件或云账号上集成测试。

阅读基线：2026-09-20。说明采用中文，界面文案建议 Bahasa Melayu 优先、English 可切换；字段、类名和命令保留英文。默认开发环境是 Windows + PowerShell，Sui CLI 可单独放在 WSL Ubuntu。所有数据示例均为虚构数据。

每一阶段如何推进

1. 先读目标与前置条件，按列出的路径建立文件。
2. 输入命令或逐段理解示例；出现 <...> 表示要替换的值，标为“伪代码 / 结构片段”的内容不能直接编译。
3. 完成“你来实现”的任务，然后执行本页验收。失败时先定位这一层，不同时改 UI、数据库和硬件。
4. 保存截图、请求响应、测试输出与简短说明；每完成一阶段建立一个 Git commit。

项目依据与优先级

本书依据你提供的 RFID/NFC flowchart、sequence diagram，以及工作区 planning/ 下的架构、数据模型和测试计划编写。原始 Mermaid 是流程依据；上次 Archify 图是摘要，省略了若干分支，不能据此宣称完整需求已经覆盖。

现有 planning/07_FIGMA_UI_SPEC.md 中的 8-10px 圆角已被你后来的要求覆盖：本书采用红白主色，约 90% 容器直角。二维码、卡片类别和上链时机的实现假设在第 02、24、29 章说明。

最终你应交付什么

- 访客手机登记；四类表单；Penjaga MRN 校验或明确标记的演示适配器。
- Staff 审核、扫码发卡、归还、逾期；Admin 库存、用户、设置和报表。
- 本地 USB 读卡服务；服务端强校验；MySQL 事务与防重复处理。
- Sui Testnet 审计证明、验证页面、可恢复 worker；线上部署与备份恢复演练。

每个“通过”都应有证据。真实医院 API、真实门禁控制与生产数据使用尚需医院提供接口或确认规则；演示适配器不等同于医院集成。

01 全局路线与里程碑

把项目分成可演示的小成果。下面的顺序优先暴露数据库、云部署与硬件兼容性问题，UI 和 Sui 在稳定接口上逐步加入。时间是学习预算，不是工期保证。

阶段	章节	结束时的成果
A 基础环境	03-09	仓库、数据库、API 健康检查、首次云端连通
B 业务核心	10-18	登录、登记、MRN adapter、审核、发卡、归还
C UI 建设	19-23	手机表单与工作人员操作界面完整闭环
D 硬件接入	24-28	真实卡片读数、安全上传、部署后的扫卡
E 区块链	29-34	Move 测试、Testnet 发布、outbox、证明验证
F 上线验收	35-42	部署、CI、故障恢复、UAT、答辩证据

建议的工作节奏

1. 第一轮只用假数据、Mock Reader 和 Mock Hospital API 跑通流程，但页面必须显示模拟模式。
2. 第二轮替换为真实 USB reader，保留相同的 ScanResult 数据契约。
3. 第三轮接入 Sui；暂停 worker 时，核心登记与发卡仍然可用。
4. 第四轮在真实手机、柜台电脑和云端完成端到端验收。

每周保留一次“从干净数据库重新搭建”的演练。建议每次只同时开发两个任务：一个主功能，一个风险验证。把硬件试读和空项目部署安排在第一周，不要等 UI 全部完成。

最早完成的三个实验

- Spring Boot 成功读取 MySQL，重启后数据保留。
- 手机通过 HTTPS 打开 /register；不是手机访问开发电脑的 localhost。
- 读卡器在 Windows 下读出同一张测试卡的稳定 UID；记录卡型、驱动与读卡模式。

验收：你可以画出每个组件的运行位置，并解释为什么 USB 读卡程序不会部署到 Railway。

02 架构、边界与必须记录的决策

推荐结构：React 在浏览器，Spring Boot 提供业务 API，MySQL 保存状态；柜台电脑运行 Reader Agent；TypeScript worker 将 outbox 中的证明提交到 Sui Testnet。访客和工作人员不需要持有 Sui 钱包。

TEXT

```
Visitor / Staff browser -> Web UI -> /api -> Spring Boot
      |
      v
Local USB reader -> Reader Agent -> HTTPS -> MySQL
      |
      audit outbox (pending)
      |
      Sui worker -> Testnet
```

三个实现决策

1. 上链时机：原始 sequence 在 Sui 回应后才确认成功；现有架构计划采用异步 outbox。本书推荐异步：业务事务提交后允许交付，审计状态显示 Pending。记录 ADR-001。若导师要求同步，必须增加“已预留 / 待证明”状态、超时恢复和补偿，不能一直持有数据库锁等区块链。

2. 读卡传输：逻辑上仍是 WebApp 发起 ReaderService 扫描。本书生产路径采用云端 Scan Job，由本地 Agent 主动领取、回传；浏览器不直接访问 localhost。此变化避免 HTTPS 页面访问本地服务时的证书、浏览器权限和网络策略差异。记录 ADR-002。

3. 类别来源：UID 本身不含 Penjaga 等业务类别。主要演示路径使用预先写入类别的 NFC 测试卡；医院既有卡无法读写时只能由批准的 UID 库存映射获取类别，属于需求实现调整。记录 ADR-003，不能静默把未知类别猜出来。

“Issued”只代表系统发出了实体通行证，不代表医院门禁控制器已经授权开门。没有门禁厂商 API 与现场许可时，项目交付到卡片管理层为止。

你来实现：在 docs/adr/ 建立这三份决策，写清背景、选择、影响与替代方案。验收：网页显示的业务成功与证明状态分别由不同字段驱动。

03 Windows 开发环境安装

先安装 Git、JDK 21、Node.js 24 LTS、IDE (IntelliJ IDEA 或 VS Code) 、Docker Desktop 与数据库客户端。Node 24 是本指南选择的版本线；以 Vite 的 engines 和 SDK package.json 为兼容性依据。Spring Boot 在 Initializr 中选当前稳定正式版，Java 21、Maven、Jar；不要选 SNAPSHOT/Milestone。参考 [R01-R04]。

操作顺序

1. 装 JDK 后让 IDE Project SDK 与 JAVA_HOME 指向同一版本；Maven 使用仓库里的 wrapper。
2. 装 Node 后重新打开终端。确认 npm.cmd 可用；如 PowerShell 阻止 npm.ps1，可调用 npm.cmd，不必全局放宽执行策略。
3. 按 Docker 官方步骤启用 WSL 2 后端；启动 Docker Desktop 并等待引擎就绪。
4. 安装数据库客户端，先不要把应用连接成 MySQL root。

POWERSHELL

```
git --version
java -version
node --version
npm.cmd --version
docker version
docker compose version
```

这些命令分别检查版本控制、Java、前端运行时和容器引擎。docker version 应同时列出 Client 与 Server；只有 Client 通常表示引擎未启动。

固定版本

创建 docs/toolchain.md：记录日期、操作系统、JDK、Initializr 生成的 Boot 版本、Node、MySQL 镜像、Sui CLI 和 @mysten/sui 版本。提交 package-lock.json、pom.xml、Maven wrapper 与 Move.lock。生成项目时可用 latest，之后构建用 npm ci，避免每次演示自动升级依赖。

验收：所有检查通过；你能在新终端复现。此阶段还不需要装 Kubernetes、Kafka、Redis 或将数据库上线。

04 从 project structure 开始

在现有工作区内建立应用目录，保留 planning/ 和提案文件。应用模块各自管理依赖，初期无需 Nx/Turborepo。以下是目标结构，不是需要一次性写满的目录清单。

```
TEXT
FYP Dev/
  frontend/      React + TypeScript + shadcn/ui
  backend/       Spring Boot + Maven wrapper
  reader-agent/  Java 21; Windows native PC/SC
  sui-worker/    Node + TypeScript + MySQL client
  move/hsaas_audit/ Move.toml, sources/, tests/
  infra/        compose.yaml, deployment notes
  docs/
    adr/         decisions and trade-offs
    api/         endpoint contracts, sample requests
    evidence/    synthetic test evidence
    runbooks/    deploy, backup, restore, reader install
  planning/     existing plans and diagrams
  .gitignore
  README.md
```

建立方法

1. 用 IDE 或 Explorer 建立上述空目录。确认自己位于项目根目录后检查 git status；没有仓库才执行 git init。
2. README 写运行顺序：MySQL、Backend、Frontend；需要读卡再启动 Agent，需要上链再启动 Worker。
3. .gitignore 排除下列内容，但保留不含秘密的 .env.example、wrapper 和 lockfile。

```
GITIGNORE
**/node_modules/
**/target/
**/dist/
**/.env
**/.env.*
!**/.env.example
**/*.keystore
**/secrets/
**/logs/
**/build/
```

补充：不要提交数据库备份、真实访客 CSV、助记词、Reader device token、私钥或 IDE HTTP client 的私密变量文件。文件被 .gitignore 排除，不会自动撤回已经提交的秘密。

验收：git status 只出现预期文档和目录文件；仓库里的任何环境示例都不含可用凭据。

05 初始化 Spring Boot 项目

进入 <https://start.spring.io>，在浏览器生成项目。选择 Maven、Java、稳定 Boot、Jar、Java 21；Group 使用 edu.upm.hsaas，Artifact 为 visitor-pass，Package 为 edu.upm.hsaas。把解压内容放进 backend/，确保 backend/pom.xml 存在，不要再多嵌套一层。

Initializr 依赖选择

依赖	具体用途
Spring Web / Web MVC	REST controller、JSON 请求与响应
Validation	@Valid、长度和必填校验
Spring Data JPA	Entity、Repository 与事务
MySQL Driver	JDBC 连接 MySQL
Flyway Migration	按版本执行 SQL migration
Spring Security	登录、角色、CSRF、会话保护
Actuator	最小健康检查与运维指标

Boot 4 的 starter 组织与旧教程可能不同；以 Initializr 为准，不复制旧版本的完整 pom。检查 Flyway 的 MySQL 支持模块是否存在；当前依赖通常需要 org.flywaydb:flyway-mysql，由 Boot dependency management 管理版本。若坐标不匹配，以所选版本的依赖树和 [R05] 为准。

```
POWERSHELL
cd backend
.\mvnw.cmd -v
.\mvnw.cmd dependency:tree
.\mvnw.cmd test
```

用途：wrapper 下载并使用指定 Maven；dependency:tree 确认实际解析的版本。数据库还未配置时，应用上下文测试可能因为 datasource 失败；先完成第 06-07 章，再要求所有测试通过，不通过删除数据库依赖掩盖问题。

你来实现：建立 config、auth、registration、card、assignment、audit、reader、hospital、common 包；主 Application 类位于 edu.upm.hsaas 根包。先不引入 Lombok，理解构造器注入后再决定是否使用。

验收：IDE 无依赖解析错误；Maven 与 IDE 都使用 Java 21。参考 [R01]。

06 本地 MySQL 与持久化

在 `infra/compose.yaml` 建立仅用于本地开发的 MySQL 服务。下面是教学配置；密码由你在未提交的 `infra/.env` 填入，镜像版本由 `toolchain.md` 固定。应用使用独立用户，`root` 仅用于管理。

```
YAML
services:
  mysql:
    image: mysql:8.4
    environment:
      MYSQL_DATABASE: hsaas
      MYSQL_USER: hsaas_app
      MYSQL_PASSWORD: ${DB_PASSWORD}
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
    ports:
      - "127.0.0.1:3306:3306"
    volumes:
      - hsaas_mysql:/var/lib/mysql
volumes:
  hsaas_mysql:
```

操作步骤

1. 创建 `infra/.env`：设置两个不同的本地测试密码；`.env.example` 只写键名与占位值。
2. 在根目录运行 `docker compose -f infra/compose.yaml up -d`。
3. 运行 `docker compose -f infra/compose.yaml logs mysql`，等待数据库 ready。
4. 数据库客户端连接 `127.0.0.1:3306 / hsaas`，使用 `hsaas_app`。

named volume 让容器重建后保留数据。停止容器不是删除数据库；不要随手使用 `down -v`，它会删除这个项目的 volume。若本机已有 MySQL，占用 3306，可把宿主端口改为 3307，同时修改 JDBC URL。

时间与编码

数据库字符集使用 `utf8mb4`；业务时间以 UTC 保存，界面按 `Asia/Kuala_Lumpur` 显示。统一 Java Instant、数据库 UTC DATETIME(6) 约定与 worker 的毫秒格式，别混合本地时间字符串。MYSQL_* 初始化变量通常只在空数据目录首次初始化时生效；改 `.env` 不会自动修改已存在账户密码。

验收：建立一条临时测试记录，重启容器后仍存在；随后通过 migration 管理正式表。此阶段不把 3306 暴露给局域网。

07 Backend 配置与第一个 endpoint

位置：backend/src/main/resources/application.yml。以下结构片段将环境值与代码分离。Spring Boot 默认不会自动读取任意 .env 文件；在 IDE Run Configuration 或 PowerShell 进程中设置 DB_URL、DB_USER、DB_PASSWORD。

```
YAML
server:
  port: ${PORT:8080}
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USER}
    password: ${DB_PASSWORD}
    hikari:
      maximum-pool-size: 5
  jpa:
    hibernate:
      ddl-auto: validate
    open-in-view: false
  flyway:
    enabled: true
  management:
    endpoints:
      web:
        exposure:
          include: health
```

本地 DB_URL 可为 jdbc:mysql://127.0.0.1:3306/hsaas?connectionTimeZone=UTC。不要把关闭 TLS 或 allowPublicKeyRetrieval 当成通用云端修复；按数据库证书与驱动要求配置。

先做最小连通性

1. 建立 V1_baseline.sql（下一章），再加入与表匹配的 Entity。
2. 创建 GET /api/public/ping，返回 {"status":"ok"}。通过 SecurityConfig 只开放此路径和必要公共 API；不要全局 permitAll。
3. 从 backend 运行 .\mvnw.cmd spring-boot:run；访问 http://localhost:8080/api/public/ping。
4. 验证 /actuator/health 的最小响应；线上不公开 env、beans 或详细 DB 错误。

ddl-auto: validate 只检查映射，不帮你偷偷修改表；open-in-view: false 强迫你在 service 内准备 DTO，避免 controller 序列化时产生隐藏查询。

验收：HTTP 200、Flyway 成功、DB 连接正常；密码错误时日志不泄漏明文密码。参考 [R01、R05]。

08 数据模型与 migration 顺序

统一术语：本书使用 cards / card_assignments 表，对应旧计划的 passes / pass_assignments。实体卡库存、访客登记、一次借用和历史事件必须分开；不要把所有状态放进一张 visitor 表。

表	最少需要的内容
users	用户名、密码哈希、角色、active、时间戳
visitor_categories / destinations	四类 code、地点、启用状态
visitor_registrations	随机 reference、类别、资料、MRN 状态、审核者
cards	规范 UID 唯一、类别、AVAILABLE/ISSUED 等、version
card_assignments	卡、登记、issue/due/return/closed 时间、操作者
audit_events	不可覆盖的事件、canonical bytes/hash、schema version
audit_outbox	event ID、重试状态、lease、digest、确认时间
scan_jobs / scan_results	设备、操作上下文、过期时间、扫描证据
idempotency_records	用户、操作、key、请求哈希、结果 reference

建表顺序

1. V1_identity_and_reference.sql: 用户、类别、地点。
2. V2_registration_and_cards.sql: 登记与实体卡；加入 UID unique。
3. V3_assignments_and_constraints.sql: 借用记录、外键、活动分配约束。
4. V4_audit_and_outbox.sql; V5_reader_and_idempotency.sql。

文件放在 backend/src/main/resources/db/migration/。用新 migration 修改已发布 schema，不改旧 migration 的内容。为状态筛选、登记时间、卡片 UID、due_at 和 outbox 可领取条件添加索引。

注册资料初期可存 visitor_registrations 中；如提取 visitors 表，不要仅凭相同姓名合并访客。证件、电话、MRN 的收集范围与保留期仍需医院确认。演示先用虚构数据。

验收：空库启动可自动建立所有表；第二次启动无重复 seed；旧数据升级成功。保存 ERD 和 migration 日志，而不只保存数据库客户端截图。

09 约束与项目分层

MySQL 不直接提供 PostgreSQL 那样的 WHERE partial unique index。可用生成列限制 “一个卡片 / 登记只有一个未关闭分配”；示例假设 card_id 与 registration_id 为 BIGINT，closed_at 为空表示活动。

SQL

```
ALTER TABLE card_assignments
  ADD active_card_id BIGINT GENERATED ALWAYS AS
    (CASE WHEN closed_at IS NULL THEN card_id ELSE NULL END)
  STORED,
  ADD UNIQUE KEY uq_active_card (active_card_id);
```

你来实现：按同样方式为 registration_id 加活动唯一约束；加入外键、NOT NULL 和合法状态约束。多个已关闭记录生成 NULL，可继续保存历史。必须用真实 MySQL 测试；H2 行为不等同于 MySQL。参考 [R06]。

分层职责

TEXT

```
registration/
  RegistrationController.java    HTTP + @Valid
  RegistrationService.java       rules + transactions
  RegistrationRepository.java    database access
  Registration.java              JPA entity
  dto/CreateRegistrationRequest.java
  dto/RegistrationResponse.java
common/error/                   safe API errors
```

Controller 不直接实现发卡规则；Repository 不调用 Sui；DTO 不直接返回所有 Entity 字段。使用构造器注入，需要什么依赖一目了然。

第一次部署实验

在完成 ping 与 DB migration 后，提前走第 35-37 章部署一个空壳。只用虚构数据和最小接口，确认 Java 启动、PORT、MySQL 私网与 Vercel /api 转发。完成后回到业务开发。

验收：尝试插入两条同卡 active assignment，第二条被数据库拒绝；第一条关闭后允许重新分配。能够从手机访问云端 ping。

10 API 契约先于页面

在 docs/api/ 写 endpoint、角色、请求、响应和失败码，再写 Controller。下面是教学契约，最终字段由真实表单和 UI 决定。所有写操作服务端重新验证，不能相信浏览器传来的角色或卡片状态。

接口	权限 / 用途
GET /api/public/registration-config	匿名；类别、地点、表单规则
POST /api/public/registrations	匿名；创建登记
GET /api/staff/registrations	Staff/Admin；分页待处理队列
POST /api/staff/registrations/{id}/verify	记录审核结果
POST /api/staff/registrations/{id}/reject	拒绝并保存理由
POST /api/staff/assignments	以登记 ID + scanResultId 发卡
POST /api/staff/assignments/{id}/return	归还并释放库存
POST /api/admin/cards	以 scanResultId 登记实体卡
GET /api/admin/proofs	证明状态与验证入口

```
JSON
{
  "code": "CARD_CATEGORY_MISMATCH",
  "message": "The card category is not compatible.",
  "correlationId": "a-random-request-id",
  "fieldErrors": []
}
```

HTTP 400 表示请求格式错误；401 未登录；403 权限/CSRF 不通过；409 重复 UID、资源状态冲突；422 可用于合法格式但不符合业务条件。为项目选择一致策略，记录而不是随意混用。服务异常可用 503，并避免将 SQL 或 stack trace 返回给前端。

你来实现：@RestControllerAdvice、错误 code enum、分页与排序白名单；选择与 Boot 兼容的 springdoc 版本后生成 OpenAPI。未知排序字段不可直接拼入 SQL。

验收：用 IDE HTTP Client 或 Postman 保存至少一组成功和失败请求；响应不包含 password_hash、完整审计内部字段或数据库地址。

11 登录与角色：先选一种方式

本书选择服务端 Session + HttpOnly Cookie + CSRF，浏览器统一请求同源 /api。开发用 Vite proxy，部署用 Vercel rewrite 转发到 Railway。这样不用自己实现 JWT refresh/rotation。旧计划中的 JWT_SECRET 在这条路线中不需要。

建立用户与登录

1. users 表保存 PasswordEncoder 生成的哈希，使用 UserDetailsService 从数据库读取用户、active 和角色。
2. 使用 Spring Security 自带认证流程，配置 /api/auth/login 作为 formLogin processing URL；请求是 application/x-www-form-urlencoded。
3. successHandler 返回最小用户资料 JSON；failureHandler 返回 401 JSON；不要依赖默认 HTML redirect。
4. GET /api/auth/me 返回当前身份；POST /api/auth/logout 失效 session 并清 cookie。

JAVA

```
// Structure inside SecurityFilterChain configuration
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/public/**", "/api/auth/csrf",
        "/actuator/health").permitAll()
    .requestMatchers("/api/admin/**").hasRole("ADMIN")
    .requestMatchers("/api/staff/**")
        .hasAnyRole("COUNTER_STAFF", "ADMIN")
    .anyRequest().authenticated());
```

用途：请求进入业务层之前先按路径限制角色。仍要用方法级授权或 service 内规则处理“谁能改哪个对象”。示例不是完整 SecurityConfig；你还需添加登录/登出、JSON 错误 handler、会话策略及设备专用 filter chain。

首个管理员通过一次性 bootstrap profile/命令创建：密码从环境注入、只在 users 为空时执行、创建后移除 bootstrap secret。不要在 migration 中写固定明文管理员密码。保留至少一名 active admin；禁止 Staff 调用用户管理接口。

验收：匿名 401、Staff 访问 Admin 403、禁用用户不能登录；改角色后现存会话需要失效或重新加载权限。

12 Session、CSRF 与前端调用

默认 Session CSRF 可由一个允许匿名访问的 endpoint 暴露当前请求 token。下面使用 Spring Security 提供的 `CsrfToken` 参数；返回值不要写入日志。保留默认 CSRF 保护，前端按服务器返回的 `headerName` 发送。

JAVA

```
@GetMapping("/api/auth/csrf")
Map<String, String> csrf(CsrfToken token) {
    return Map.of("headerName", token.getHeaderName(),
                  "token", token.getToken());
}
```

TYPESCRIPT

```
const csrf = await fetch('/api/auth/csrf', {
  credentials: 'include'
}).then(r => r.json());

await fetch('/api/auth/login', {
  method: 'POST', credentials: 'include',
  headers: { [csrf.headerName]: csrf.token },
  body: new URLSearchParams({ username, password })
});
```

这是认证调用片段；username/password 来自表单。必须检查 `response.ok`，不能把 401 当作登录成功。登录与登出后重新获取 CSRF token，清理用户缓存；所有 POST/PATCH/DELETE 也应带 token，包括匿名登记写入。参考 [R07]。

线上 Session cookie：HttpOnly、Secure、Path=/、SameSite=Lax，通常不设置 Domain；本地 HTTP profile 可关闭 Secure。若 Vercel 代理后 cookie、Location 或 HTTPS 推断错误，检查第 37 章。不要通过关闭 CSRF 来解决所有 403。

单实例原型可用内存 Session，但重启会退出登录。正式演示如要求跨重启会话，加入 Spring Session JDBC，按所选版本官方 schema 用 Flyway 管理 session 表，并测试过期清理。

验收：缺少 CSRF 的写请求被拒绝；正确 token 能提交；退出后旧 session 不可发卡；浏览器无法读取 Session cookie。

13 四类访客登记：后端先完成

注册类别固定为 EXECUTIVE、PENJAGA、VENDOR、CONTRACTOR，显示名称由 UI 翻译。具体字段以医院批准表单为准；以下姓名、电话、单位、目的等只是建议 schema，不能声称已确认。

实现顺序

1. GET config 返回启用类别、地点、字段规则和版本号，不返回内部权限设置。
2. 创建 CreateRegistrationRequest: common 字段 + category-specific 对象；@Valid 做长度/格式，service 做类别条件校验。
3. PENJAGA 检查当前 MRN verification 是否对应本次 MRN + ward，并未过期；其他类别不接受多余患者资料。
4. 生成随机 publicReference，保存 SUBMITTED，返回 reference 与提交状态。公开状态页只展示最低限度信息。

```
JAVA
// DTO fragment: add your validated category-specific fields
public record CreateRegistrationRequest(
    @NotBlank @Size(max = 120) String fullName,
    @NotNull VisitorCategory category,
    @NotNull Long destinationId,
    @NotBlank @Size(max = 32) String phone,
    String mrnVerificationToken
) {}
```

用途：DTO 限制 API 输入边界；它不是 JPA Entity，不包含 verifiedBy 或 status 让匿名访客任意指定。不要把电话当整数，保留国际前缀与前导零。证件规则按证件类型分别验证，不用单一“12 位数字”限制所有人。

你来实现：必要告知/确认文案、字段白名单、请求体大小限制、重复提交 key、限速、错误字段映射。Public reference 不应是简单递增 ID；如果能查询敏感资料，还需额外授权，随机 ID 本身不是完整权限控制。

验收：四类有效请求、空值、超长、未知类别、停用地点、重复提交、伪造 VERIFIED 均有预期结果。

14 Hospital API adapter 与 MRN 校验

你目前没有确认可用的医院 API 合同与凭据。先定义 adapter，把 mock 与真实实现隔离；对外返回 VALID、INVALID、UNAVAILABLE，不能把超时等同于“病人不存在”。

```
JAVA
public interface HospitalGateway {
    MrnCheckResult validate(String mrn, String wardCode);
}
// MrnCheckResult is your own result DTO.
// Implement MockHospitalGateway first.
// Implement ApprovedHospitalApiGateway after receiving a contract.
```

从 mock 到真实 API

1. Mock 固定三个测试案例：匹配、不匹配、超时。不要用“任何非空 MRN 都成功”的假校验。
2. 后端接收 MRN/ward，调用 adapter；设连接与响应超时，记录 correlationId，不记录完整患者资料。
3. 后端保存校验摘要，返回短时 token；把 token 绑定 session/nonce、MRN 指纹、ward、结果、过期时间。
4. 最终 POST registration 再校验 token；更改 MRN 或 ward 后必须重新验证，不能相信前端绿色标记。
5. 获得接口后确认认证、请求字段、结果码、限流、数据最小化和测试环境；用 contract tests 替换 mock。

原始 flowchart 要求 Penjaga 校验通过才继续。本书默认严格模式：INVALID 返回修改，UNAVAILABLE 保留表单并提示重试。若医院批准人工审核，可允许提交为 MRN_PENDING，但 Staff 在完成有记录的人工审核前不可标记 VERIFIED。这个模式变化须记录，不能自动降级为 VALID。

配置建议：HOSPITAL_MODE=mock|real；真实 URL 和凭据只在 Backend，前端不直接请求医院网络。

验收：三个结果都可演示；输入被改动后旧 token 失效；UI 明显显示 Mock mode；错误提示不会帮助匿名用户批量枚举患者。

15 队列、审核与拒绝

目标：让 Staff 明确知道“已提交”和“已验证”不同；分配卡片只能从 VERIFIED 进入。队列可先每 5 秒刷新，后续如有必要再引入 SSE。不要先把“real-time”当成必须搭 WebSocket。

后端步骤

1. 实现分页 GET registrations?status=SUBMITTED&page=0&size=20；限制最大 page size。
2. Detail endpoint 按权限返回必要资料，列表默认遮罩电话与证件。
3. verify command 在事务中读取当前状态，检查 MRN 条件，保存 verifiedBy、verifiedAt、规则版本和 audit event。
4. reject command 只允许指定状态，必须给 reason；已发卡的登记不能直接改为 REJECTED。

TEXT

```
SUBMITTED -> VERIFIED
SUBMITTED -> REJECTED
VERIFIED -> assignment allowed (if no active assignment)
REJECTED -> no assignment
```

你来实现：一致的状态转换函数；两名 Staff 同时审核时返回已处理或 409；身份来自认证上下文，不来自 JSON 里的 actorId。

UI 的实际行为

选择一条队列记录后展示详情、类别、目的地、MRN 来源与状态。审核按钮旁明确说明审核的是访客资料；确认后界面才出现 Assign Access Card。被拒绝时保留列表与理由，不能当作网络错误无限 Retry。

验收：直接绕过 UI 调用 assignment API，SUBMITTED、REJECTED 都无法发卡；新登记在约定刷新时间内出现；两名审核者不会覆盖对方的审计记录。

16 卡片入库与发卡事务

扫描仪证明“某个设备在某个上下文读取过这些字节”；最终资格判断由 Backend 完成。Admin 入库提交 scanResultId；Backend 取可信扫描结果、验证类别、规范 UID、尝试插入。UID 必须数据库唯一，避免两个请求同时通过 exists 检查。

```
JAVA
// Service structure; helper methods are for you to implement.
@Transactional
public AssignmentResponse issue(IssueCommand command, Actor actor) {
    var registration = registrations.lock(command.registrationId());
    var scan = scans.lockUsable(command.scanResultId(), actor);
    var card = cards.lockByUid(scan.uid());
    rules.requireVerifiedAndCompatible(registration, card, scan);
    var assignment = assignments.create(registration, card, actor);
    card.markIssued();
    scans.consume(scan);
    audit.appendAndEnqueue(assignment, actor);
    return mapper.toResponse(assignment);
}
```

逐行理解

@Transactional 把这些数据库写操作绑定为一次提交。lock 方法由你用 JPA PESSIMISTIC_WRITE / SELECT FOR UPDATE 实现。require... 检查登记 VERIFIED、无 active assignment、卡存在且 AVAILABLE、类别兼容、scan 未过期且属于当前用户/操作。consume 防止同一扫描被再次利用。audit... 同一事务写审计与 outbox，不在此处调用网络。

规定统一加锁顺序并应用到所有相关命令，例如 registration、scan、card、assignment；归还或后台操作不能逆序抢同一组锁。并发冲突返回可识别 409，数据库活动唯一约束是最后防线。

验收：两名 Staff 同时给不同访客分同一卡，只有一个成功；另一个收到 CARD_NOT_AVAILABLE。中途写审计失败时，assignment、card status、scan consumed 和 outbox 全部回滚。

17 幂等性、归还与逾期

禁用按钮只能减少误点，不能保证网络重试不会发两次卡。每个写入命令使用 Idempotency-Key，并按 actor + operation + key 建唯一记录；同 key 不同请求体返回 409。

实现幂等命令

1. 前端开始一次操作时生成 UUID，重试沿用；新操作才生成新 key。
2. Backend 在同一事务中争抢 idempotency record，保存 request hash 与最终 resource ID。
3. 已完成的同请求返回原结果；进行中的请求返回约定的可重试状态，不再创建第二个 assignment。
4. 事务回滚时不留伪“完成”记录；保留时长与业务重试窗口一致。

归还与库存

TEXT

```
Card AVAILABLE -> ISSUED -> AVAILABLE
                        ^
Card ISSUED -> OVERDUE -----+ (returned)
Card ISSUED / OVERDUE -> LOST (authorised action)
Card AVAILABLE -> DISABLED (administration)
```

RETURNED 是事件；成功归还后库存为 AVAILABLE。事务锁定相关记录，设置 returnedAt、returnedBy、closedAt，释放活动唯一约束，再写 RETURNED audit/outbox。无法读卡时按授权人工查找并记录原因，不伪造扫描。

逾期 job 按 due_at < now UTC 分批处理；以 assignment ID + OVERDUE 事件唯一性防重复，更新状态前重新锁定检查是否已归还。到期时间在发卡时保存快照；后来改配置不能悄悄改旧借用期限。

验收：重复归还不会增加两条事件；归还后的卡可再次借出；调时钟或用测试 Clock 模拟逾期；scheduler 与归还同时发生后数据库状态一致。

18 审计、报表与管理员功能

把业务状态、操作历史与链上证明分开。audit_events 是事件快照；audit_outbox 是送往 Sui 的投递状态；UI 不要把“Pending proof”显示成“发卡失败”。

按以下顺序实现

1. 用户管理：创建、禁用、角色变更；保护最后一个 active admin；变更写审计。
2. 库存管理：类别、卡状态、丢失/停用；正在使用的卡不能直接删掉。
3. 地点和规则：优先停用而非删除被历史引用的数据；保存设置版本。
4. Dashboard：当日登记数、待审核数、活动借用数、逾期数。
5. Audit viewer：按事件/操作者/日期筛选，查看 Pending/Confirmed/Failed 与本地记录。

SQL

```
SELECT c.code, COUNT(*) AS registrations
FROM visitor_registrations r
JOIN visitor_categories c ON c.id = r.category_id
WHERE r.registered_at >= :startUtc
      AND r.registered_at < :endUtc
GROUP BY c.code;
```

这是参数化查询思路，:startUtc / :endUtc 由应用绑定。半开区间可避免连续日期重复计数。不要用 SUM 当前库存状态推断历史某天访客数；历史报表从登记、assignment 和事件时间计算。

你来实现：统一站点时区边界；报表权限；导出默认遮罩；CSV 单元格以 =、+、-、@ 开头时处理公式注入；筛选与总计共用条件。

验收：准备固定数据集，如 4 类各 3 次登记，共 12 条；查询总数与图表一致。修改历史数据的实验必须在隔离测试库执行，用于第 34 章的篡改检测。

19 初始化 React、Vite 与 shadcn/ui

从根目录建立 frontend，先完成默认页面再加组件。下面选择 React + TypeScript，Tailwind 使用 Vite 插件路线，避免把旧 Tailwind 3 教程与新版本混在一起。参考 [R02、R08]。

POWERSHELL

```
npm.cmd create vite@latest frontend -- --template react-ts
cd frontend
npm.cmd install
npm.cmd install tailwindcss @tailwindcss/vite
npm.cmd install -D @types/node
```

配置步骤

1. 在 vite.config.ts 保留 React plugin，加入 tailwindcss plugin；@ alias 指向 src。用 fileURLToPath(new URL('./src', import.meta.url)) 避免 ESM 下 __dirname 问题。
2. 在 tsconfig.json 与 tsconfig.app.json 的对应 compilerOptions 中设置 paths: {"@/*":["./src/*"]}, 按 CLI 检查结果补齐 baseUrl。
3. src/index.css 导入 @import "tailwindcss"; main.tsx 引入该 CSS。
4. 运行下面命令初始化并添加实际需要的 UI 组件。

POWERSHELL

```
npx.cmd shadcn@latest init
npx.cmd shadcn@latest add button input label select
npx.cmd shadcn@latest add table dialog alert badge tabs
npx.cmd shadcn@latest add sheet skeleton sonner
npm.cmd install react-router-dom @tanstack/react-query
npm.cmd install react-hook-form zod @hookform/resolvers
npm.cmd run dev
```

shadcn CLI 将组件代码放进项目；组件能由你修改，不是安装后自动生成整个医院 UI。初始化选项随版本可能变化，以实际 CLI 提示为准，并提交锁文件。

验收：Button 与 Input 有样式；@/components/ui/button 可解析；npm run build 无 TypeScript 错误。先不复制网上完整 dashboard。

20 前端目录、路由与请求层

TEXT

```
frontend/src/
  app/                router.tsx, providers.tsx
  components/ui/       shadcn generated components
  components/          PageHeader, StatusBadge, ScanPanel
  features/
    auth/              login, session hooks
    registration/      mobile forms, queue, review
    cards/              inventory, add-card flow
    assignments/       issue, return, overdue
    audit/              proof list, verifier
  lib/                 api.ts, errors.ts, dates.ts
  styles/              tokens.css
```

一层一层接入

1. 根节点放 QueryClientProvider; Router 建立 PublicLayout 与 StaffLayout。
2. /register、/register/success 为公共页面; /staff/* 与 /admin/* 依据 /api/auth/me 展示。
3. api.ts 统一 /api 前缀、credentials、CSRF header、JSON 和安全错误 code。Session 失效时清除 Query cache 并回登录。
4. 列表数据使用 TanStack Query; 表单草稿使用组件状态或 React Hook Form。不要同时在多个地方维护同一份服务器库存。

TYPESCRIPT

```
// Add inside Vite's defineConfig object.
server: {
  proxy: {
    '/api': { target: 'http://localhost:8080', changeOrigin: true }
  }
}
```

用途：浏览器访问 :5173/api, Vite 代为连接 :8080, 开发时同源 cookie 更容易保持一致。它只在开发服务器有效；线上用第 37 章的 rewrite。

写操作成功后 invalidate 相关 query, 如 registrations、cards、assignments、summary。路由保护仅改善 UX, 后端仍必须授权。不要把 MRN、IC 或电话放在 URL query 或 analytics event。

验收：手动输入 /admin/users 时 Staff 被挡住，同时直接 HTTP 调用仍为 403；刷新任一子路径可正常打开。

21 红白直角设计系统

按你的最新要求：白色工作区、深红主操作，约 90% 容器与表格直角；少量头像或状态点可圆形。红色不应用于每一块内容，成功/等待/错误必须有文字区分。

```
CSS

:root {
  --background: #fafafa;
  --foreground: #171717;
  --card: #ffffff;
  --primary: #9d0b0f;
  --primary-foreground: #ffffff;
  --border: #e5e5e5;
  --radius: 0rem;
}

@theme inline {
  --radius-sm: 0px;
  --radius-md: 0px;
  --radius-lg: 0px;
  --radius-xl: 0px;
}
```

在 shadcn 生成的 tokens 基础上调整，不删除其余语义变量。新版本可能有更多 radius token，逐项覆盖；不要用全局 * {border-radius:0} 破坏 radio 和头像。主色是项目建议值，不代表已取得官方品牌规范。

页面构建顺序

1. AppShell：左侧导航、顶部当前角色/柜台、内容区；主要区域用分隔线而不是层层 Card。
2. PageHeader：标题、说明、主操作；每页只有一个主要动作。
3. FilterBar + Table：搜索、类别、状态、分页；再加详情抽屉。
4. ScanPanel：设备状态、提示、结果、错误、重试；UID 单独可复制。

手机约 390px 单列，桌面约 1440px 队列/详情双栏；表单控件触摸区域至少约 44px；键盘 focus、label、错误说明和对比度均需检查。Dialog 打开后焦点进入、关闭后回原按钮，不能只靠颜色解释状态。

验收：做一页组件展示，核对 Button/Input/Table/Dialog/Alert 的正常、禁用、loading、error 状态；大部分容器呈直角。参考 [R08]，布局灵感可用 shadcn Studio 与 NameThatUI，但不直接复制付费资产。

22 访客手机 UI：从二维码到 reference

目标是一个连续的手机流程：类别、资料、确认提交、reference。先构建 EXECUTIVE 最小表单，再把另外三类做成基于类别的字段 schema；不要建立四份完全重复的表单。

操作步骤

1. Counter QR 页面生成 `https://<web-domain>/register?counter=<public-code>`。后端验证 counter code 是否启用；它不授予工作人员权限。
2. `/register` 请求 config，显示四类选项、语言与隐私说明。
3. PENJAGA 增加 MRN/ward 校验区；显示 Checking/Valid/Invalid/Unavailable，修改字段后清除旧成功状态。
4. 提交时冻结同一 idempotency key；网络未知结果时先查询最小提交结果，再决定重试。
5. 成功页显示 reference、等候柜台指示；不展示完整证件或患者资料。

TYPESCRIPT

```
// UI validation fragment; server repeats these rules.
const commonSchema = z.object({
  fullName: z.string().trim().min(1).max(120),
  phone: z.string().trim().min(7).max(32),
  category: z.enum([
    'EXECUTIVE', 'PENJAGA', 'VENDOR', 'CONTRACTOR'
  ])
});
```

你来实现：用 discriminatedUnion 或条件规则为类别增加字段；resolver 连接 React Hook Form；字段错误放在控件旁；取消/返回保留内存草稿。不要默认把含 MRN/证件草稿长期写 localStorage。

QR 可由你选择的二维码库在前端生成。扫描使用手机原生相机打开 URL，网页本身无需申请摄像头权限。打印时同时写短网址与柜台说明。

验收：真实手机完成四类表单；MRN 失败可修改；断网不丢失当前输入；重复点击不产生两个 reference；手机 URL 不是 localhost。

23 Staff / Admin UI 与扫描状态机

按业务优先做页面：Login、Queue + Review、Assign、Return/Overdue、Display QR、Inventory + Add Card、Users/Settings、Audit/Verifier、Analytics。空数据、权限不足、网络失败和 loading 都是完整页面的一部分。

TYPESCRIPT

```
type ScanState =
  | { kind: 'idle' }
  | { kind: 'waiting'; jobId: string }
  | { kind: 'read'; scanResultId: string; uid: string }
  | { kind: 'error'; code: string; canRetry: boolean }
  | { kind: 'submitting'; idempotencyKey: string }
  | { kind: 'done'; assignmentId: string };
```

用途：明确界面当前允许的动作。用 reducer 实现转换，避免多个 isScanning/isSuccess/isError 同时为 true。扫码成功只是进入 read，不意味着后台已发卡成功。

分配面板行为

1. 先固定当前 verified registration 和柜台设备，再创建 scan job。
2. waiting 显示 tap 提示与取消；设备离线时不给用户假倒计时。
3. read 展示 UID、可信类别与校验状态；确认提交只发 scanResultId，不让用户改 UID/category。
4. 409 显示对应错误与 Retry，保留访客上下文；Retry 创建新 job，旧 job 失效。
5. done 显示实体交付指示；Pending audit 为次级状态。归还用独立面板与确认文字。

Admin 添加卡片采用相同 ScanPanel，但 purpose=REGISTER_CARD，成功后库存刷新。重复 UID 明确“已存在”，不能自动覆盖类别。切换访客、关闭面板、退出登录时取消当前 job，并忽略迟到的旧响应。

验收：未登记、不可用、类别不匹配、读卡失败、超时、重复 UID 都有独立可理解提示；键盘能完成全部操作。

24 硬件准备：先确认卡，再选 reader

默认讲解 Windows + USB PC/SC reader；ACS ACR1252U 是可参考的 PC/SC NFC 读写器，不代表你已有或必须购买这个型号。采购前借用一台兼容设备测试医院样卡。参考 ACS 产品与 API 文档 [R09-R10]。

清单与兼容性

- 一台 Windows 柜台电脑、USB reader、厂家驱动与工具。
- 至少 6 张测试卡：四类有效卡、未入库卡、无效/空白类别卡。
- 手机用于打开 QR；打印纸/标签；备用 USB 线或端口。
- 记录频率、协议、卡片家族、UID 是否稳定、存储是否可读写。

13.56MHz NFC reader 不能因此读取任意 125kHz 门禁卡；支持 ISO 14443 也不代表能解开医院加密应用区。NTAG 系列适合测试 NDEF，但不等于医院实际通行卡。卡片外观相同不表示技术相同。

类别如何从卡片取得

原始需求是“UID 和 category 自动读取，操作员不手输”。满足它需要预先配置：在批准的可写测试卡中存 schemaVersion + category code，可选 profile signature；UID 由 reader 获取。日常入库/发卡只读，不现场选类别。

若医院卡只提供 UID：由医院批准的卡片清单映射类别，扫码后 Backend 查映射；此时应把图中的文字改为“读取 UID，系统解析类别”。这是一条有条件替代路径，不能说类别来自物理 UID。

UID 不是密码，可能被复制、模拟或随机化；读到 UID 不能证明卡不可伪造。本项目用它作库存标识，并通过人工作业与后台规则管理通行证。

验收：填写 docs/hardware-matrix.md，逐卡记录读 UID/类别能力。任何未知类别均阻止入库，不能自动设为 Penjaga。

25 Windows 驱动与独立试读

先用厂家工具验证读卡，再写 Java。Reader Agent 运行在 Windows 宿主机；数据库容器、WSL 和 Railway 都不能自动看到这台电脑的 USB 设备。

操作步骤

1. 从厂商官网下载匹配型号、系统架构的驱动；设备管理器应出现 Smart card readers。
2. 检查 Windows Smart Card 服务与厂家工具能识别 reader。先关闭其他占用同一卡片的工具。
3. 打开厂商 APDU utility，选择正确 PICC/contactless 接口；部分设备还有 SAM slot。
4. 按该型号手册执行只读 UID 命令。记录完整响应与 status word，不修改医院卡片。
5. 重复 tap/remove 20 次，比较 UID 格式、长度和一致性。

JAVA

```
// Diagnostic fragment, inside a method; Java 21.
var terminals = TerminalFactory.getDefault().terminals().list();
if (terminals.isEmpty()) throw new IllegalStateException("NO_READER");
var terminal = terminals.get(0); // choose by configured name in agent
if (!terminal.waitForCardPresent(15000)) return;
var card = terminal.connect("*");
try {
    var reply = card.getBasicChannel().transmit(
        new CommandAPDU(new byte[]{
            (byte) 0xFF, (byte) 0xCA, 0, 0, 0}));
    if (reply.getSW() != 0x9000) throw new IllegalStateException("READ_FAILED");
    String uid = HexFormat.of().withUpperCase().formatHex(reply.getData());
} finally { card.disconnect(false); }
```

导入 javax.smartcardio.* 与 java.util.HexFormat。FF CA 是相关 reader 的伪 APDU，用于获取 UID，不能当作所有卡/reader 的通用命令，也不会返回业务类别。getSW 检查 9000；getData 去掉状态字。选择 reader、异常映射和输出处理由你补全。参考 [R10-R11]。

验收：厂家工具与 Java 得到相同 UID；无 reader、超时、拔出、错误 slot 都能解释。不要用 ATR 代替 UID。

26 卡片 provisioning 与类别解析

本页是“测试卡预配置”的一次性流程，不属于 Staff 日常发卡。只对你拥有或获准配置的测试卡进行；未知医院卡保留只读。

预配置步骤

1. 选择已验证支持 NDEF 的 NFC 测试卡；定义唯一记录类型，例如 application/vnd.hsaas.card。
2. 用兼容 NDEF writer 写入最小 payload，记录规范文档与工具版本。
3. 移开并重新 tap，读取并解析 NDEF，确认字节与类别一致。
4. 如要求发现类别被篡改，用离线 provisioning key 对 version、UID、category、cardProfileId 的规范字节签名；Backend 验签。签名可能超过小容量卡片空间，先量容量，再决定卡型。

JSON

```
{
  "v": 1,
  "profileId": "random-card-profile-id",
  "category": "PENJAGA"
}
```

这是 NDEF 应用数据示例，不是把 JSON 直接发送给 APDU。你需要 reader/card adapter 读取用户区，解析 TLV / NDEF records、类型、长度和编码，再校验 JSON schema；具体读页命令与内存边界必须按卡型手册实现。遇到不支持、截断、未知 v 或多条冲突记录时失败。

Backend 信任规则

入库时检查类别是否在允许集合；有签名时校验 UID 绑定；无签名的演示卡仅能说明读取能力，不能宣称类别不可篡改。发卡时数据库里的登记类别是权威来源；卡上类别与库存不一致应阻止，并引导 Admin 调查，不能自动更新库存。

你来实现：CardProfileReader 接口、NdefCardProfileReader、MockCardProfileReader，以及未知版本/坏编码/缺字段测试。先用静态 NDEF bytes 测 parser，再接硬件。

验收：四类卡自动显示正确类别，空白卡和被修改类别卡被识别；日常 Add Card 与 Assign 不出现手输 UID/category 输入框。

27 Reader Agent 与云端 Scan Job

创建独立 Java 21 reader-agent 工程，可用 Maven；普通 Java main + HTTP client + PC/SC 即可，不必再启一个完整 Web 应用。以 deviceId 绑定柜台；USB Agent 主动通过 HTTPS 连接 Backend。

TEXT

```
Browser -> POST /api/staff/scan-jobs
Backend -> save WAITING job (user, device, purpose, expiry)
Agent -> POST /api/device/scan-jobs/claim
Agent -> wait for physical tap; read UID + profile
Agent -> POST /api/device/scan-jobs/{id}/result
Browser -> GET /api/staff/scan-jobs/{id}
Backend -> READY + scanResultId
```

Admin 创建 REGISTER_CARD job 时使用独立管理员路径或 service 权限检查。设备 token 只能领取/完成自己的 job，不能创建访客、发卡或读取姓名/MRN。Spring Security 为 /api/device/** 使用独立 stateless token 认证链，设备请求不依赖浏览器 cookie；CSRF 排除仅限该链。

你要实现的 job 规则

1. 一个 device 同时只有一个 active job；claim 必须原子，保存 lease/claim nonce。
2. 结果绑定 job、device、purpose、用户和目标 registration；设置例如 30 秒扫描期限与短时结果有效期。
3. 切换访客或取消后旧 job 拒绝新结果；job ID 随机且服务端检查归属。
4. Agent 不保存访客资料；未联网不能宣称发卡成功。只传 UID、profile、reader metadata 与必要时间信息。
5. 最终业务 command 在数据库事务中消费 result；发卡失败的重试策略要明确，不能复用已消费证据。

验收：另一柜台无法领取此 job；复制 scanResultId 到另一登记被拒绝；过期、取消和重复结果不会引发发卡。

28 Agent 打包与现场安装

安装顺序

1. 先完成 MockReader + HTTPS job 流，再替换 PC/SC adapter。
2. 将 reader-agent 打包为 Jar 或 jpackage 应用；自带 runtime 时包含 java.smartcardio 模块。
3. 每个工作站单独注册 deviceId 和高熵 device token；后端只保存 token 校验值，支持撤销/轮换。
4. device token 放 Windows 凭据存储或限制 ACL 的配置中，不放 Git、不显示在 UI。
5. 用 Windows Task Scheduler 建启动/登录后运行任务，避免重复启动两个实例；设置失败重启与本地无敏感日志。

PROPERTIES

```
# Local agent configuration example; token stored separately.
api.baseUrl=https://your-api-domain
device.id=counter-01
reader.name=YOUR_VERIFIED_PCSC_READER_NAME
scan.timeoutSeconds=30
heartbeat.intervalSeconds=10
```

Heartbeat 保存设备版本、连接状态、最近在线时间；超过例如 30 秒标为离线，这是建议阈值，按网络实测调整。读到一张卡后等待移除再允许下一次结果，避免卡片一直放在 reader 上产生重复事件。

现场检查

- 开机无需 IDE 即可上线；拔插 reader 后可恢复。
- 浏览器为 Vercel HTTPS、Agent 为本机程序；二者通过 Backend job 协调。
- USB 插到另一端口后 reader 名称变化时有明确诊断；不自动随意选择 SAM slot。
- 云端断网显示“无法扫描 / 重试”，不把上次 UID 当新卡。

如果仍选择 browser -> localhost 方案，需额外实现 origin 白名单、设备配对、短时 challenge、本地 TLS/浏览器权限与跨版本验证；CORS 不是身份认证。本书采用云端 job，因而不需给浏览器开放本地 HTTP 端口。

验收：用新 Windows 用户/重启场景运行 Agent、完成入库和发卡；设备被撤销后立刻不能提交结果。

29 Sui 的职责与证明数据

Sui 负责提供事件哈希的外部可验证记录；MySQL 负责访客身份、状态和业务查询。哈希只能证明某个承诺值与后来提供的数据是否一致，不能证明最初的登记真实、访客本人身份或系统从未遗漏事件。

定义 proof v1

- 随机 eventId; payloadVersion; payloadHash (32 bytes) ; eventType; 可选发生时间。
- 不公开姓名、IC、电话、MRN、ward、UID-to-person 映射。
- canonical payload 与随机 nonce 保留在受控数据库；链上只放承诺哈希。

TEXT

```
CanonicalAuditV1 (example ordered fields)
schemaVersion, eventId, eventType, occurredAtUtcMs,
assignmentRef, actorRef, beforeStatus, afterStatus, nonce

payloadHash = SHA-256(UTF-8(canonicalBytes))
```

这是一份你要正式制定的序列化协议，不是随意 JSON.stringify 的同义词。定义字段顺序、null、字符串转义、数值范围、UTC 毫秒与 Unicode 规则。推荐成熟 canonical JSON 方案或严格固定格式，保存生成时的确切 bytes。

不要只哈希一个低熵 MRN：它可能被枚举。nonce 应由安全随机源产生，保存在链下，与事件内容共同进入哈希。nonce 公开后仍可验证，但别默认公开含个人资料的 canonical payload。

你来实现：Java AuditCanonicalizer、固定测试向量、Java/Node 对同一 bytes 算出同一 hash。验证历史时从不可变事件快照重建，不从不断变化的 current card row 重建。

验收：换行、时区、null、中文、字段顺序均有测试；改事件一个字节就得到不同 hash。记录同步/异步差异的 ADR-001。

30 安装 Sui CLI 与 Testnet 钱包

Sui 工具链变化较快，按官方安装与 onboarding 文档操作并记录精确版本。本书示例使用 WSL Ubuntu 的 Bash；不要把 Bash 命令直接粘贴到 PowerShell。Java reader 继续运行在 Windows。参考 [R12-R14]。

操作步骤

1. 如选择 WSL，按 Microsoft 文档安装 Ubuntu 并完成首次用户设置；已有环境不用重复安装。
2. 从官方 suiup README 安装对应平台工具，检查下载来源；安装 Testnet 工具链。
3. 运行 sui client 完成初始配置，明确选择 Testnet；生成专用于 FYP 的地址。
4. 到官方 faucet 页面申请 Testnet SUI；不要为此购买主网币。

```
BASH
suiup install sui@testnet
sui --version
sui client
sui client envs
sui client switch --env testnet
sui client active-address
sui client gas
```

如果 envs 没有 testnet，按当前 CLI 的 new-env --help 添加官方 Testnet endpoint。当前工具的参数可能变化，应先查看帮助；网页 faucet 是更稳定的指导入口，不能假设每个 CLI 版本都支持 Testnet faucet 命令。

钱包角色

分开 publisher（部署/升级）和 writer（审计提交）更清晰；最小 FYP 可先用一个专用低余额 Testnet 地址，后续再拆分权限。UpgradeCap 不应随便交给在线 worker；worker 只需要 AuditWriterCap 与 gas。

助记词/私钥离线保存。不要把 .sui 目录复制进项目或截图；需要配置 worker 时，按官方 keytool 流程安全导出指定测试 key，避免写入终端历史/日志。

验收：active env 为 Testnet、地址正确、gas 非零；docs/toolchain.md 记录 CLI 版本，不记录密钥。

31 Move package: 先设计再写合约

在 move/ 下运行 `sui move new hsaas_audit`, 进入新目录; 保留生成的 `Move.toml`、锁文件及当前 edition。不要复制多年以前的 dependency revision。目标是一个小型审计 package, 不需要 NFT 或代币经济。参考 [R14、R15]。

需要自己实现的三个类型

1. `AuditWriterCap`: 拥有 UID 的 capability; 仅授权地址持有, 可绑定 registry ID。
2. `Registry`: 共享对象, 含 `eventId -> hash` 的 Table; 使超时后能确定是否已写入。
3. `AuditAnchored`: `eventId`、`payloadHash`、`version` 等最小事件; 不含访客资料。

MOVE

```
// Signature sketch inside your audit module; not a full module.
public fun anchor(
    cap: &AuditWriterCap,
    registry: &mut Registry,
    event_id: vector<u8>,
    payload_hash: vector<u8>,
    payload_version: u64,
    ctx: &mut TxContext,
) {
    // Check capability-to-registry binding and input lengths.
    // If ID exists: require the same hash, then return.
    // Otherwise store the proof and emit AuditAnchored.
}
```

用途: cap 限制谁能写; registry 提供持久查重; 相同 ID + 相同 hash 可视为已经完成, 不重复 emit; 相同 ID + 不同 hash 必须 abort。事件本身不提供唯一约束, 不能只 emit 就声称幂等。

你来实现: init 创建并 share Registry、转移 cap; 固定错误码; 校验 16-byte UUID / 32-byte SHA-256 (若采用文本 UUID 就另定长度); table contains/borrow/add; event::emit。时间若来自客户端是声明时间, 不是链验证的事实; 链确认时间另行保存。

验收: 无 cap 不能调用、错误 registry 失败、重复同 hash 不新增证明、重复不同 hash abort。小规模共享 Registry 适合 FYP; 不要宣称无限吞吐量。

32 Move 测试与发布

本地测试顺序

1. 正常提交：Registry 中出现 eventId/hash，事件字段正确。
2. 输入边界：空 ID、错误 hash 长度、未知 payloadVersion 被拒绝。
3. 幂等：两次同 ID/hash 无重复 proof；同 ID 不同 hash abort。
4. 权限：不同地址无法使用不属于自己的 capability；registry 绑定不符失败。
5. 使用 sui::test_scenario 多事务场景验证共享对象和对象归还，销毁测试对象避免资源未消费错误。

BASH

```
# Run inside move/hsaas_audit
sui move build
sui move test
sui client active-env
sui client active-address
sui client publish --help
sui client publish
```

publish 交易会使用 Testnet gas。确认当前 CLI 支持该调用形式；如提示 gas budget，按模拟/官方建议设置，不把旧教程数字当永久配置。记录发布结果中的 package ID、Registry ID、AuditWriterCap ID、UpgradeCap ID、digest 与 network。

发布后配置

将 writer capability 转移到 worker 专用地址（如果分开钱包），再查询确认 ownership。package ID 和 object ID 可放 deployment manifest；私钥不放。每次重发或升级都更新 manifest，旧事件的 proof 仍记录原 package/network。

你来实现：docs/deployments/testnet.json 记录公开配置；一个使用固定虚构 eventId/hash 的 smoke test。示例合约尚未在本指南中编译，必须以你完成的代码通过 move build/test 后才发布。

验收：工具可查询已部署 package 和对象；一次 smoke 交易执行成功；事件字段无 PII；保留测试结果与公开交易链接。

33 TypeScript Sui worker 与 outbox

在 sui-worker/ 执行 `npm init -y`, 安装 `@mysten/sui`、`mysql2`; 开发依赖 `typescript`、`tsx`、`@types/node`。
`package.json` 设置 `type=module`; `tsconfig` 使用 `NodeNext`, 脚本 `dev=tsx src/main.ts`、`build=tsc`、`start=node dist/main.js`。提交 `package-lock.json`。

TYPESCRIPT

```
import { SuiGrpcClient } from '@mysten/sui/grpc';
import { Transaction } from '@mysten/sui/transactions';
import { Ed25519Keypair } from '@mysten/sui/keypairs/ed25519';

// env and proof are your validated config / outbox DTO.
const client = new SuiGrpcClient({
  network: 'testnet', baseUrl: env.SUI_GRPC_URL
});
const signer = Ed25519Keypair.fromSecretKey(env.SUI_PRIVATE_KEY);
const tx = new Transaction();
tx.moveCall({
  target: `${env.PACKAGE_ID}::audit::anchor`,
  arguments: [tx.object(env.WRITER_CAP_ID),
    tx.object(env.REGISTRY_ID),
    tx.pure.vector('u8', proof.eventIdBytes),
    tx.pure.vector('u8', proof.hashBytes),
    tx.pure.u64(proof.version)]
});
```

这是交易构建片段, 假设第 31 章签名一致; `ctx` 由 Sui 提供, 不作为普通实参。event ID 编码必须与合约一致; 64 字符 hex hash 要解码为 32 bytes, 不能把 hex 字符直接当哈希字节。

Worker 循环

短数据库事务领取可执行行 (FOR UPDATE SKIP LOCKED + lease); 提交 claim 后才调用 Sui, 避免网络等待占数据库锁。单 worker 串行提交先满足 FYP; 以后并行要处理 capability、gas 与共享对象竞争。DB 用户只给读取事件和更新 outbox 所需权限。

启动时验证 env、network、派生地址与预期地址一致、cap ownership、余额; 缺配置直接退出。VITE_* 中永远没有私钥。

验收: 先用 fake gateway 将 PENDING 跑成模拟结果 (明确标记), 再用真实 Testnet; 不能把模拟 CONFIRMED 混进真实证据。参考 [R16-R18]。

34 上链确认、重试与篡改验证

TYPESCRIPT

```
const result = await client.signAndExecuteTransaction({
  signer, transaction: tx, include: { effects: true }
});
if (result.FailedTransaction) {
  throw new Error('CHAIN_EXECUTION_FAILED');
}
if (!result.Transaction) throw new Error('UNEXPECTED_RESPONSE');
await client.waitForTransaction({ result });
const digest = result.Transaction.digest;
```

用途：Promise resolve 不等于 Move 执行成功；必须区分成功和 FailedTransaction。这段按核对时的 SDK API 编写，仍需与你锁定的版本做 TypeScript 编译与 Testnet 集成测试。参考 [R16-R17]。

恢复协议

PENDING -> PROCESSING (带 lease) -> SUBMITTED (已有 digest) -> CONFIRMED。明确失败可退回 PENDING + nextAttemptAt；超过次数进入 FAILED。RPC 超时但可能已执行时进入 RECONCILING，先查已保存 digest 或 Registry eventId，不能直接生成新 eventId。

worker 崩溃后 lease 到期可接管；用 lease token 条件更新，旧 worker 不可覆盖新 worker 结果。重试保留同一 eventId/hash；指数退避加 jitter。执行前若 Registry 已有相同 proof，可确认已有记录；digest 暂时不可恢复时标记“proof 已查到，交易引用待恢复”，不要伪造 digest。

Verifier 页面

1. 从 audit event snapshot 和 nonce 重新生成 canonical bytes/hash。
2. 查询记录的 network + package + registry 下 eventId，取得链上 hash。
3. 展示 MATCH、MISMATCH、NOT_ANCHORED 或 UNAVAILABLE；链节点不可达不能显示 MISMATCH。
4. 如使用 event 验证，还应核对 package/module/event type、发送者和交易成功状态。

验收：隔离测试库中修改事件内容，验证变为 MISMATCH；停 worker 后业务成功且 proof Pending；恢复后补交；同事件重复运行没有第二条不同证明。

35 部署拓扑与环境变量

线上建立四个位置：Vercel frontend；Railway backend、MySQL、sui-worker；柜台本地 reader-agent；Sui Testnet package。仅 Backend 有公共业务 API，MySQL 使用私网，Worker 无需公共端口。参考 [R19-R21]。

位置	你要配置的值
Frontend	/api 相对路径；公开的站点名与语言
Backend	DB_URL/USER/PASSWORD、profile、Session、医院 API、设备认证配置
Worker	DB 凭据、SUI_GRPC_URL、PRIVATE_KEY、EXPECTED_ADDRESS、package/cap/registry IDs
Reader Agent	API HTTPS URL、deviceId、单独 device token、reader name
Cloud platform	构建目录、启动命令、PORT、healthcheck、region、secret storage

本地 / test / prod 三套配置

- 1. Local：虚构数据、Mock Hospital 可用、真实 reader 可选。
- 2. Test：独立数据库、专用 Testnet wallet/object、禁用真实个人资料。
- 3. Demo/prod-like：明确 mock 标记、稳定域名、最少日志、备份与演示数据。

不要把 Railway MySQL URL 直接当 JDBC URL。Backend 需要 jdbc:mysql://host:port/database；worker mysql2 使用对应 host/port/user/password/database，配置方式不同。

server.port 使用 \${PORT:8080} 是 Spring application.yml 的占位语法；Railway Variables 的服务引用使用平台自己的语法。不要把两种语法混抄。

上线前核对平台当前计划、用量计费、额度和限制，不假设免费资源可以一直运行。此指南没有替你创建账号、开通计费、发布 Move 或部署任何服务。

验收：画出每个 secret 所在位置；能解释为什么 frontend、worker 和 reader 各自不能获得不必要的凭据。

36 Railway: Backend、MySQL、Worker

首次部署步骤

1. 建立 Railway project 和测试环境，添加 MySQL；选择与本地接近的版本/配置，确认持久化卷。
2. 添加 Git 仓库 Backend service，Root Directory=/backend；从平台选择构建方案或自写多阶段 Dockerfile。
3. 引用 MySQL 服务变量组成 DB_URL，使用私网 host；设置 prod profile。迁移账号与应用账号生产化时分开，FYP 至少不要让业务日常使用 root。
4. 启动 java -jar 指定 Jar，绑定平台 PORT，配置 /actuator/health；只公开不含细节的健康响应。
5. 看 migration、启动和健康日志；成功后生成 HTTPS 域名，先测 ping 再测 DB。
6. 建 Worker service，Root Directory=/sui-worker；build=npm ci && npm run build，start=npm start；添加 worker secrets，不开 public domain。

DOCKERFILE

```
# Dockerfile fragment: build stage must produce a known app.jar.
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /out/app.jar app.jar
USER 10001
ENTRYPOINT ["java", "-jar", "app.jar"]
```

你来实现的 build stage：JDK 21、复制 pom/wrapper 与 src、运行 ./mvnw package、把确定名称的可执行 Jar 复制到 /out/app.jar。不要用可能匹配多个 Jar 的 COPY target/*.jar。容器构建需 wrapper 的 Unix 执行权限和 LF 换行。

本地首次 docker build 后运行 smoke，再推到 Railway。不要为了构建成功永久 skip tests；CI 先测试，部署产物与测试 commit 对应。

验收：重启 Backend 可重新连接数据；Worker 能读取 outbox；MySQL 无公开入口；错误凭据导致明确失败，健康接口不输出 DB 地址。参考 [R19-R20]。

37 Vercel、同源 API 与 cookie

在 Vercel 导入仓库，Framework=Vite，Root Directory=frontend，Build=npm run build，Output=dist。浏览器请求 /api；下面配置先匹配 API 再兜底 React Router。参考 [R21]。

```
JSON
{
  "rewrites": [
    {
      "source": "/api/:path*",
      "destination": "https://YOUR-API-DOMAIN/api/:path*"
    },
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

这只是 frontend/vercel.json 片段，替换域名。部署后必须验证真实代理行为，尤其 Set-Cookie、POST、无缓存与错误响应。不要把包含个人资料的 GET、CSRF 或登录响应设 public cache；后端返回 Cache-Control: no-store。

浏览器检查顺序

1. 打开 /register，再直接刷新 /staff/registrations；不应 404。
2. 请求 /api/auth/csrf，应为 JSON 而不是 SPA HTML。
3. 登录并查看 cookie：HttpOnly、Secure、host-only、Path=/；后续 /api/auth/me 仍有身份。
4. 发卡 POST 带 CSRF，通过代理到 Backend；登录/登出没有跳转到 localhost。
5. 检查 HTTPS forwarding。若需要 Spring forwarded headers，只信任所用平台代理路径，防止任意客户端伪造来源。

Preview 和 Production 使用各自 Backend/DB；rewrite 的域名不能随便指到真实环境。若不用代理而直接跨域，必须独立设计 CORS、cookie SameSite/第三方 cookie 限制与 origin 白名单，这不是只改一个 URL。

验收：真实手机扫描公网 QR；柜台电脑打开同一部署，Agent 仍走 Backend 公网 HTTPS；登记、审核、读卡、发卡闭环可用。

38 CI、版本发布与回滚

目标：同一 commit 可以在新机器构建，不依赖你的 IDE 缓存。CI 使用隔离测试数据库和测试 secrets，不访问真实医院数据。

依次建立 pipeline jobs

1. Backend: Java 21 + Maven cache; ./mvnw verify; 集成测试使用 MySQL Testcontainers 或 CI MySQL service。
2. Frontend: 固定 Node, npm ci, lint、组件测试、npm run build。
3. Worker: npm ci、TypeScript build、fake gateway 单元测试; Testnet smoke 单独手动触发。
4. Move: 固定 Testnet 兼容 CLI 版本, sui move build/test。
5. Reader: 编译、parser/mock transport 测试; 真实 USB 测试标为手动硬件 gate。

CI 不自动发布链上 package，也不把每次 PR 都接入同一个共享演示 Registry。版本 manifest 记录 git commit、镜像/构建标识、migration 版本、Sui package/registry、Agent 版本。

发布与回滚策略

先向后兼容加字段、部署能够兼容新旧 schema 的应用，再迁移数据，最后清理旧字段。应用回滚不等于数据库 migration 回滚；Flyway 已执行 SQL 不能靠回退 Git 自动撤销。失败时优先 forward fix，涉及恢复则走下一章。

你来实现：docs/runbooks/release.md 写一份发布单：测试通过、备份完成、migration 预演、环境校验、部署、smoke、监控窗口、回退条件。

验收：从新 clone 可构建；坏 migration 阻止上线；UI 与 API 版本不兼容时有清楚处理；同一版本的证据和部署 manifest 可追溯。

39 备份恢复、日志与故障处理

数据库备份需要实际恢复验证。先使用平台可用的备份能力，再补充受控逻辑备份；不要承诺某个计划一定包含自动备份。文件含个人资料时按项目访问规则保存，不提交仓库。

BASH

```
# Example run on an authorised machine with DB connectivity.
mysqldump --host=DB_HOST --user=BACKUP_USER -p \
  --single-transaction --no-tablespaces \
  --result-file=hsaas-backup.sql hsaas
```

这是 Bash 示例。--result-file 避免某些 shell 重定向编码问题；-p 交互输入密码。--single-transaction 主要适用于 InnoDB，一致性窗口内避免 schema 变更。备份过程与用户权限需按数据库版本测试。

恢复演练

1. 暂停测试环境 Worker，建立独立恢复库，绝不覆盖唯一原库。
2. 使用 mysql 客户端的 SOURCE 或批准的 restore 工具导入备份。
3. 检查行数、外键、最近事件、active assignments、用户权限与 Flyway history。
4. 恢复出来的 outbox 可能早于链上状态；重新连接原 network/registry 后先 reconcile，避免重复提交。
5. 验证完成后才恢复服务；记录恢复时间和可能丢失的时间窗口。

日志只记录 correlationId、匿名资源 ID、错误 code、耗时；不记录密码、token、MRN、完整表单或 Sui 私钥。监控 queue age、worker heartbeat、FAILED outbox、设备离线、DB pool 使用率和 API 错误率。

验收：给自己一个“主服务重启 + worker 停止 + reader 拔出”的演练，按 runbook 恢复；不需要靠记忆或临时改源码。

40 测试矩阵：按需求验收

测试	必须观察的结果
四类登记	字段规则不同，合法数据生成 reference
MRN 无效 / API 超时	可区分 INVALID 与 UNAVAILABLE，保留输入
读卡失败 / 未插 reader	可重试，不使用上次 UID
重复卡片 UID	409；库存无第二条记录
未审核登记发卡	后端拒绝，UI 绕过也无效
未登记 / 不可用 / 错类别	分别拒绝，不改变 assignment
并发发同一卡	只有一条 active assignment
重复 HTTP 命令	相同结果，不重复审计和 outbox
归还与逾期竞争	最终状态一致，事件不重复
旧扫描 / 跨柜台扫描	过期或归属不符被拒绝
Sui 断网 / 超时未知	Pending/Reconcile；核心业务不假失败
链上 Move abort	Failed，不显示 Confirmed
篡改快照 / 节点断网	Mismatch 与 Unavailable 不混淆
恢复数据库备份	数据完整，可重新对账

测试怎么写

Service 单元测试验证规则；Repository 集成测试用真实 MySQL 验证锁和 unique；HTTP tests 检查角色与 CSRF；前端测试看用户可见行为；端到端用真实浏览器。真实 USB 和实际 Testnet 各保留一组手动 smoke。

并发测试用两个独立事务和同步起跑屏障，确保请求确实重叠，而不是连续调用两次。验证数据库最终行数和状态，不能只判断其中一次 HTTP 409。

你来实现：每个案例写 Given/When/Then、数据集、预期错误码、实际结果和证据路径。性能目标由你与导师约定；报告区分 API 响应、读卡等待和链确认耗时。

验收：这张表每行都有可复现记录；不能用一张“测试通过”截图替代所有关键规则。

41 常见问题排查顺序

现象	先检查什么
Boot 启动失败	JDK/Boot 版本、env 是否传入、DB 可连接、migration 日志
Flyway checksum mismatch	旧 migration 是否被改；新增修复脚本，勿盲目 repair
API 401	Session cookie、代理、登录结果和用户 active
API 403	角色、CSRF 是否刷新、路径命中哪条 filter chain
API 返回 HTML	/api rewrite 是否被 SPA fallback 抢先匹配
Unknown database / timezone 错	JDBC URL、schema 初始化与 UTC 格式
shadcn 样式失效	CSS import、Tailwind plugin、alias 与 CLI 配置
手机打不开 QR	二维码公网地址、Wi-Fi/HTTPS，不能使用 localhost
Java 找不到 reader	驱动、Windows Smart Card 服务、正确 slot、是否在 WSL 运行
UID 读到但 category 为空	卡片是否预配置、协议/NDEF parser、不是 UID 自带类别
卡一直放着重复触发	缺少 remove-before-next-scan 与 job 单次消费
Sui object not found	network/package/registry/cap 是否同一部署
Sui insufficient gas	测试钱包余额、配置地址、faucet 状态
Promise 成功但链失败	是否检查 FailedTransaction 和 status
outbox 长期 PROCESSING	lease 过期回收、worker 崩溃与死循环

统一定位方法

从浏览器 Network 抄 correlationId，找到 Backend 日志，再查对应数据库状态，最后看设备或 Worker。每次只替换一个变量；先用 mock 隔离外部依赖，再重放真实请求。

不要用以下方式“修好”：全局 permitAll、关闭 CSRF、禁用数据库 unique、在 UI 写死成功、把所有 reader 错误映射成同一张测试卡、直接修改 outbox 为 CONFIRMED。

验收：随机选三种故障，由另一人按 runbook 重现和恢复；说明根因而不是仅重启所有服务。

42 最终演示、交付与自主学习顺序

10 分钟演示脚本

1. 展示架构、四个部署位置与当前 Mock/Real 标记。
2. Admin tap 一张新卡入库；再 tap 同卡演示 duplicate。
3. 手机扫 QR 提交 Penjaga；展示 MRN 无效与有效路径。
4. Staff 审核；先 tap 错类别卡，再 tap 正确 AVAILABLE 卡发放。
5. 展示 assignment、库存 Issued 与 Pending proof；Worker 完成后显示 Confirmed。
6. 验证 hash 一致；在隔离测试数据上演示篡改后 mismatch。
7. 归还卡，库存回 Available；展示逾期与失败重试证据。

完成标准

- 新机器可按 README 搭建；所有环境键名有解释；无可用秘密进入 Git。
- 四类注册、审核、入库、发卡、归还、逾期、管理与报表可操作。
- Reader 可在开机后运行；设备绑定、超时、取消、重试完整。
- Move 有权限与幂等测试；Worker 有 lease、reconciliation 与错误状态。
- 公网 HTTPS、Session/CSRF、迁移、备份恢复、最小权限有验收记录。
- 真实医院 API 未接入就明确说明；门禁自动开门不在无接口条件下宣称完成。

交付文件：README、ERD、API 契约、ADR、测试报告、UAT 表、部署 manifest、Reader 安装说明、用户手册、备份恢复 runbook、已知限制。导师讨论时用这些材料解释实现选择。

你下一次应该先做什么

先完成第 03-07 章：目录、工具链、Initializr、MySQL、ping 和 health。然后拿你的 pom.xml、目录树、启动结果与遇到的问题继续逐步实现。本书保留的 helper、Entity、Controller 和业务细节就是你要自己编写和理解的部分。

A 代码片段索引与补全任务

章节 / 片段	放在哪里	仍需你实现
04 .gitignore	仓库根目录	项目专属敏感路径与 README
06 compose.yaml	infra/	密码文件、连接检查、备份
07 application.yml	backend resources	profile、认证、环境注入
09 generated column	Flyway migration	活动登记约束、FK、测试
11 security rules	backend/config	认证 provider、JSON handlers、设备链
12 CSRF controller/client	auth + frontend/lib	错误处理、token 刷新、Session
13 request record	registration/dto	四类字段、条件校验、mapper
14 HospitalGateway	hospital/	mock、真实 adapter、token 验证
16 issue service	assignment/	锁查询、规则、审计、回滚测试
18 aggregate query	analytics repository	授权、时区、分页与导出
19-21 UI setup	frontend/	alias、provider、tokens、各页面
23 ScanState	features/cards	reducer、API、取消/过期逻辑
25 PC/SC fragment	reader-agent/	终端选择、异常、移卡与适配器
26 NDEF payload	测试卡 / parser fixtures	卡型读写、schema、可选签名
31 Move signature	move/.../sources	类型、init、table、权限、测试
33-34 SDK snippets	sui-worker/src	配置、数据库 claim、恢复、验证
36 Docker fragment	backend/Dockerfile	build stage、Jar 名称、smoke
37 rewrites	frontend/vercel.json	环境域名、cookie 检查、禁缓存

本书代码是示例与结构片段，不是一套可直接复制运行的完整项目。命令可按注明目录执行，但你仍需安装依赖、替换占位值、处理 imports 与补齐类型。用于教学的伪代码通过文字说明标记。

建议每实现一个 helper，就写一句说明“输入是什么、检查什么、写哪些数据、失败会怎样”。能解释这些，比背完整源码更适合你的 FYP。

B 官方参考与版本核对 (1)

以下为本书核对过的官方入口；具体安装版本、平台 UI 与依赖坐标会变化。资料核对日期为 2026-09-20。网页用于工具/API 事实核对，HSAAS 的业务规则与实现建议来自你的需求和本书设计。

[R01] Spring Boot system requirements; Initializr 生成的版本与兼容性优先。

<https://docs.spring.io/spring-boot/system-requirements.html>

<https://start.spring.io/>

[R02] Vite getting started; 核对 Node engines、React TypeScript 模板。

<https://vite.dev/guide/>

[R03] Docker Desktop for Windows; 核对 Windows / WSL 安装前置条件。

<https://docs.docker.com/desktop/setup/install/windows-install/>

[R04] Microsoft WSL basic commands; Sui Bash 环境可选路径。

<https://learn.microsoft.com/en-us/windows/wsl/basic-commands>

[R05] Flyway MySQL support; 核对与 Boot 对应的数据库支持模块。

<https://documentation.red-gate.com/fd/mysql-277579322.html>

[R06] MySQL 8.4 manual; generated columns、unique indexes、InnoDB locking。

<https://dev.mysql.com/doc/refman/8.4/en/create-table-generated-columns.html>

<https://dev.mysql.com/doc/refman/8.4/en/create-index.html>

[R07] Spring Security CSRF; Session、token endpoint 与 SPA 集成。

<https://docs.spring.io/spring-security/reference/servlet/exploits/csrf.html>

[R08] shadcn/ui Vite installation; CLI、alias、Tailwind 配置。

<https://ui.shadcn.com/docs/installation/vite>

[R09] ACS ACR1252U product; 能力、驱动与型号文档入口。

<https://www.acs.com.hk/en/products/342/acr1252u-usb-nfc-reader-iii-nfc-forum-certified-reader/>

[R10] 从 R09 的 Application Programming Interface 下载该型号手册；查询 UID 指令、读页命令、状态字与卡型限制。具体硬件尚未确认，手册为选型参考，不是兼容性证明。

[R11] Java 21 javax.smartcardio; 终端枚举、连接与 APDU。

<https://docs.oracle.com/en/java/javase/21/docs/api/java.smartcardio/javax.smartcardio/package-summary.html>

C 官方参考与版本核对 (2)

[R12] Sui CLI installation; suiup 与平台安装方式。

<https://docs.sui.io/getting-started/onboarding/sui-install>

[R13] Sui faucet; 使用 Testnet 测试币。

<https://faucet.sui.io/>

[R14] Sui build and publish onboarding; build/test/publish 工作流。

<https://docs.sui.io/getting-started/onboarding/hello-world>

[R15] Sui framework source / reference; 按实际编译版本查 table、event、transfer、TxContext API。

<https://docs.sui.io/references/framework>

[R16] Mysten TypeScript SDK gRPC client; 采用 @mysten/sui/grpc。

<https://sdk.mystenlabs.com/sui/clients/grpc>

[R17] Executing transactions; 检查 FailedTransaction 和确认读取。

<https://sdk.mystenlabs.com/sui/clients/executing>

<https://sdk.mystenlabs.com/sui/transactions/signing-and-execution>

[R18] Transaction input reference; 纯值、对象、u8 vector 与 u64。

<https://sdk.mystenlabs.com/sui/transactions/reference>

[R19] Railway Spring Boot deployment; 项目根、构建与启动。

<https://docs.railway.com/guides/spring-boot>

[R20] Railway MySQL; 连接变量、服务与持久化。

<https://docs.railway.com/databases/mysql>

[R21] Vite on Vercel; 构建配置与 SPA deep-link rewrite。

<https://vercel.com/docs/frameworks/frontend/vite>

工作区项目依据：planning/README.md、02_TECHNICAL_ARCHITECTURE.md、03_REQUIREMENTS_AND_TEST_PLAN.md、04_DATA_MODEL_AND_API_PLAN.md、07_FIGMA_UI_SPEC.md、08_MERMAID_DIAGRAMS.md，以及你的原始 Mermaid 和最新红白直角设计要求。

本书将不确定项明确留在对应步骤：医院表单/MRN 合同、实际 reader/card 型号、类别 provisioning、门禁厂商接口、云端预算。它们不会阻止你先完成 mock 和接口层，但决定真实集成能否验收。